



✓ [Mảng 1 Chiều Nâng Cao]. Bài 7. Sắp xếp chẵn lẻ

[C](#)[C++ CB NC](#)[C++ NC](#)[DSA](#)[JAVA OOP](#)[View
as PDF](#)

Cho một mảng các số nguyên, sắp xếp các phần tử trong mảng sao cho, các phần tử lẻ đứng trước và giảm dần, các phần tử chẵn đứng sau và tăng dần. Xem thêm ví dụ để hiểu rõ hơn yêu cầu.

[Submit solution](#)

Đầu vào

Dòng đầu tiên là số lượng phần tử trong mảng.

Dòng thứ 2 là **N** phần tử trong mảng.

Giới hạn

$$1 \leq n \leq 10^6$$

$$1 \leq a_i \leq 10^9$$

Đầu ra

In ra dãy đã được sắp xếp

Ví dụ :

Input 01

```
10
1 2 3 9 7 4 8 6 10 5
```

[Copy](#)

Output 01

[Copy](#)[My submissions](#)[All submissions](#)[Best submissions](#)

✓ **Points:** 1.00 (partial)

🕒 **Time limit:** 1.0s

📄 **Memory limit:** 256M

🖨 **Input:** stdin

🖨 **Output:** stdout

📝 **Problem source:**
28Tech

➤ **Problem type**

✓ **Allowed languages**
C, C#, C++, Java, Kotlin,
Pascal, PyPy, Python,
Scratch



Comments

[Report an issue](#)

Please read the [guidelines](#) before commenting.

commented 5:21:11 pm, 21/09/2025  

tuong_706

include <stdio.h>

```
int partition (int a[], int l, int r){ int pivot = a[r]; int i = l-1; for (int j=l; j<r; j++){ if (a[j] <= pivot){ ++i; int tmp=a[i]; a[i]= a[j]; a[j]= tmp; } } ++i; int tmp = a[i]; a[i] = a[r]; a[r] = tmp; return i; }
```

```
void quickSort (int a[], int l, int r){ if (l < r){ int p = partition(a, l, r); quickSort(a, l, p-1); quickSort(a, p+1, r); } }
```

```
void tachmang(int a[], int n){ int chan[n], le[n]; int j=0, k=0; for (int i=0; i<n; i++){ if (a[i] %2==0){ chan[j] = a[i]; j++; } else { le[k] = a[i]; k++; } } quickSort(chan, 0, j-1); quickSort(le, 0, k-1); int cnt = k-1; for (int i=0; i<k; i++){ a[i] = le[cnt]; cnt--; } cnt=0; for (int i=k; i<n; i++){ if (cnt==j) break; a[i] = chan[cnt]; cnt++; } }
```

```
int main(){ int n; scanf("%d", &n); int a[n]; for (int i=0; i<n; i++){ scanf("%d", &a[i]); } tachmang(a, n); for (int i=0; i<n; i++){ printf("%d ", a[i]); } return 0; }
```

commented 9:58:47 am, 08/04/2025 

KHOA87

MẠNH THỊ



-2
▼



dungdeptrai

AC FULL

Ý tưởng là dùng mergeSort, tách mảng ban đầu thành 2 mảng toàn số chẵn và mảng toàn số lẻ. Dùng mergeSort cho từng mảng rồi printf 2 mảng đó là xong...

```
include <stdio.h>  
include <math.h>
```

```
//giam dan
```

```
void merge(int a[], int l, int m, int r){
```

```
    int n1 = m-l+1, n2 = r-m;  
    int x[n1], y[n2];  
    for(int i = 0; i<n1; i++){  
        x[i] = a[l+i];  
    }  
    for(int i = 0; i<n2; i++){  
        y[i] = a[m+i+1];  
    }  
    int index = l;  
    int i = 0, j = 0;  
    while(i<n1 && j<n2){  
        if(x[i] >= y[j]){  
            a[index++] = x[i++];  
        } else {  
            a[index++] = y[j++];  
        }  
    }  
    while(i<n1){  
        a[index++] = x[i++];  
    }  
    while(j<n2) {  
        a[index++] = y[j++];  
    }  
}
```

Copy

```
}
```



```
if(l<r){
    int m = (l+r)/2;
    mergeSort(a, l, m);
    mergeSort(a, m+1, r);
    merge(a, l, m, r);
}
```

[Copy](#)

```
}
```

```
// tang dan
```

```
void merge1(int a[], int l, int m, int r){
```

```
    int n1 = m-l+1, n2 = r-m;
    int x[n1], y[n2];
    for(int i = 0; i<n1; i++){
        x[i] = a[l+i];
    }
    for(int i = 0; i<n2; i++){
        y[i] = a[m+i+1];
    }
    int index = l;
    int i = 0, j = 0;
    while(i<n1 && j<n2){
        if(x[i] <= y[j]){
            a[index++] = x[i++];
        } else {
            a[index++] = y[j++];
        }
    }
    while(i<n1){
        a[index++] = x[i++];
    }
    while(j<n2) {
        a[index++] = y[j++];
    }
}
```

[Copy](#)

```
}
```



Hello, Nguyễn Bảo Toàn.

```
if(l<r){  
    int m = (l+r)/2;  
    mergeSort1(a, l, m);  
    mergeSort1(a, m+1, r);  
    merge1(a, l, m, r);  
}
```

Copy

}

```
int main(){
```

Copy



```
scanf("%d", &n);
int a[n];
int indexOdd = 0;
int indexEven = 0;
for(int i = 0; i < n; i++){
    scanf("%d", &a[i]);
    if(a[i] & 1){
        ++indexOdd;
    } else {
        ++indexEven;
    }
}
int b[indexEven]; // mang luu gia tri chan
int c[indexOdd]; // mang luu gia tri le
int k = 0, j = 0;
for(int i = 0; i<n;i++){
    if(a[i] & 1){
        c[j++] = a[i];
    } else {
        b[k++] = a[i];
    }
}

mergeSort(c, 0, indexOdd-1);
mergeSort1(b, 0, indexEven-1);

for(int i = 0; i<indexOdd; i++){
    printf("%d ", c[i]);
}
for(int i = 0; i<indexEven; i++){
    printf("%d ", b[i]);
}

return 0;

}
```



tranminhnhat_isagi110



Hello, Nguyễn Bảo Toàn.

làm 2 cái vector chẵn lẻ rồi sắp xếp là đc mà , làm j mà phức tạp thế

^
0
v

**dungdeptrai**

commented 6:26:44 am, 30/08/2025



cái này dùng c :) chứ dùng c++ nó nhàn hơn

^
0
v

**yenchiiu**

commented 3:39:31 pm, 05/08/2025



cần gì đâu bạn cứ tách chẵn lẻ ra 2 mảng khác nhau mảng lẻ thì sort thêm cmp $x > y$ thôi còn cái mảng chẵn thì sort 1 lần xong cout ra 2 mảng đó theo đề bài thôi cần gì đến merge

^
-1
v

**dungdeptrai**

commented 6:30:20 am, 30/08/2025



Trong c không có cái sort như c++...

^
-10
v

**longhk2012**

commented 8:38:58 am, 08/09/2024



This comment is hidden due to too much negative feedback. [Show it anyway.](#)



Hello, Nguyễn Bảo Toàn.

Post!

Được phát triển bởi **28TECH.COM.VN**